# ステートマシン構造解説

本ドキュメントは、`StateMachineAI`名前空間のステートマシンシステムがどのように構築され、どこでどの処理を行っているかを解説します。

---

## 目次


1. システム全体の構成
2. 各コアクラスの処理内容
3. 実装例の解説
4. 処理の流れ（ライフサイクル）
5. 変数命名規則


---

## システム全体の構成

このステートマシンは**3つのコアクラス**で構成されています：

| ファイル | 役割 |
|---|---|
| [StateMachine.cs](file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/StateMachine.cs) | ステートの管理・実行エンジン（心臓部） |
| [State.cs](file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/State.cs) | 各ステートの基底クラス（状態の雛形） |
| [StatefulObjectBase.cs](file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/StatefulObjectBase.cs) | MonoBehaviour統合用の抽象クラス（接続役） |

### クラス関係図

```
classDiagram
    class StateMachine~T~ {
        -State~T~ m_CurrentState
        -Stack~State~T~~ m_StateHistory
        -State~T~ m_GlobalState
        +event OnStateChanged
        +ChangeState()
        +RevertToPreviousState()
        +Update()
    }

    class State~T~ {
        +T owner
        #State~T~ m_ParentState
        #StateMachine~T~ m_SubStateMachine
        #List~State~T~~ m_SubStates
        +Enter()
        +Stay()
        +Exit()
        +AddSubState()
        +ChangeSubState()
    }

    class StatefulObjectBase~T,TEnum~ {
        +List~State~T~~ stateList
        #StateMachine~T~ stateMachine
        +ChangeState()
        +Update()
    }
```

```
    StateMachine --> State : 管理
    StatefulObjectBase --> StateMachine : 所有
    StatefulObjectBase --> State : リスト保持
    State --> StateMachine : サブステート用
```

---

## 各コアクラスの処理内容

### State<T> - ステートの基底クラス

**ファイル**: [State.cs](file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/State.cs)

各ステート（状態）の**テンプレート**です。

#### 主要プロパティ

| プロパティ | 型 | 説明 |
|---|---|---|
| `owner` | `T` | このステートを利用するオブジェクト |
| `m_ParentState` | `State<T>` | 親ステート（サブステートの場合のみ使用） |
| `m_SubStateMachine` | `StateMachine<T>` | サブステート用のステートマシン |
| `m_SubStates` | `List<State<T>>` | サブステートのリスト |

#### ライフサイクルメソッド

| メソッド | 呼び出しタイミング | Unity相当 |
|---|---|---|
| `Enter()` | ステート開始時に**1回だけ** | `Start()` |
| `Stay()` | ステート中**毎フレーム** | `Update()` |
| `Exit()` | ステート終了時に**1回だけ** | デストラクタ |

#### サブステート関連メソッド

| メソッド | 説明 |
|---|---|
| `AddSubState(State<T>)` | サブステートを追加 |
| `ChangeSubState(int)` | インデックス指定でサブステートに遷移 |
| `ChangeSubState(State<T>)` | ステート直接指定で遷移 |
| `HasSubStates()` | サブステートを持っているか確認 |
| `UpdateSubStates()` | サブステートを更新 |
| `GetParentState()` | 親ステートを取得 |

---

### StateMachine<T> - ステートマシン本体

**ファイル**: [StateMachine.cs](file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/StateMachine.cs)

ステートの**管理・実行エンジン**です。

#### 主要メンバー変数

| 変数名 | 型 | 説明 |
|---|---|---|
| `m_CurrentState` | `State<T>` | 現在実行中のステート |
| `m_StateHistory` | `Stack<State<T>>` | ステート履歴スタック |
| `m_MaxHistoryCount` | `int` | 履歴の最大保持数（デフォルト: 10） |
| `m_GlobalState` | `State<T>` | グローバルステート（常に実行） |

**主要メソッド**

| メソッド | 処理内容 |
|---|---|
| `ChangeState(state, saveHistory)` | 現在ステートの`Exit()`→履歴保存→新ステートの`Enter()` |
| `Update()` | グローバル→現在→サブの順に`Stay()`を実行 |
| `RevertToPreviousState()` | 履歴スタックから前のステートを取り出して遷移 |
| `SetGlobalState(state)` | 全ステート共通で実行される処理を設定 |
| `ClearHistory()` | 履歴をクリア |

**ChangeState の処理詳細**

```
public void ChangeState(State<T> state, bool saveHistory = true)
{
    State<T> previousState = m_CurrentState;

    if (m_CurrentState != null)
    {
        if (saveHistory)
        {
            m_StateHistory.Push(m_CurrentState);  // ① 履歴に保存
            TrimHistory();                        // ② 最大数超過分を削除
        }
        m_CurrentState.Exit();                    // ③ 終了処理
    }

    m_CurrentState = state;                       // ④ 新しいステートに変更
    m_CurrentState.Enter();                       // ⑤ 開始処理

    OnStateChanged?.Invoke(previousState, m_CurrentState);  // ⑥ イベント通知
}
```

**Update の処理詳細**

```
public void Update()
{
    // グローバルステートを先に実行
    if (m_GlobalState != null)
        m_GlobalState.Stay();

    // 現在のステートを実行
    if (m_CurrentState != null)
    {
        m_CurrentState.Stay();

        // サブステートがあれば更新
        if (m_CurrentState.HasSubStates())
            m_CurrentState.UpdateSubStates();
    }
}
```

---

## StatefulObjectBase<T, TEnum> - MonoBehaviour統合

**ファイル**: [StatefulObjectBase.cs] (file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/StatefulObjectBase.cs)

Unityの`MonoBehaviour`とステートマシンを**橋渡し**する抽象クラスです。

**主要メンバー**

| 変数/メソッド | 説明 |
|---|---|
| `stateList` | 登録されたステートのリスト（Enumの順番と一致させる） |
| `stateMachine` | 内部のステートマシンインスタンス |
| `Update()` | 毎フレーム`stateMachine.Update()`を呼び出す |
| `ChangeState(TEnum)` | Enum指定でステート遷移 |

#### ChangeState の処理

```
public virtual void ChangeState(TEnum state)
{
    if (stateMachine == null)
        return;

    // Enumを整数に変換してstateListのインデックスとして使用
    stateMachine.ChangeState(stateList[state.ToInt32(null)]);
}
```

> [!IMPORTANT] `Enum`の値の順番と`stateList`に追加する順番は**必ず一致**させてください。

---

## 実装例の解説

### 管理クラス（AITester）

**ファイル**: [AITester.cs](file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/AITester.cs)

```
public class AITester : StatefulObjectBase<AITester, AIState_Type>
```

#### Enumの定義

```
public enum AIState_Type
{
    Idle,       // 0: 待機
    Attack,     // 1: 攻撃
    Search,     // 2: 索敵
    Tracking,   // 3: 追跡
    Hit,        // 4: 被弾
    Die,        // 5: 死亡
    A_Mode,     // 6: テストモードA
    B_Mode,     // 7: テストモードB
}
```

#### 初期化処理（AISetUp）

```
public void AISetUp()
{
    // ① ステートマシンを生成
    stateMachine = new StateMachine<AITester>();

    // ② グローバルステートの設定（常に実行される監視役）
    SetGlobalState(new S_GlobalMonitor(this));

    // ③ ステートの登録（Enumの順番で追加）
    stateList.Clear();
    stateList.Add(new S_Idle(this));    // 0: Idle
    stateList.Add(new S_Attack(this));  // 1: Attack
    // ... 以下続く
```

```
    // ④ ステート変更イベントの購読
    SubscribeStateChanged((prev, next) =>
    {
        Debug.Log($"[StateChanged] {prev} -> {next}");
    });

    // ⑤ 初期ステート設定
    ChangeState(AIState_Type.Idle);
}
```

---

### シンプルなステート（S_Idle）

**ファイル**: [S_Idle.cs](file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/S_Idle.cs)

```
public class S_Idle : State<AITester>
{
    public S_Idle(AITester owner) : base(owner) { }

    public override void Enter()
    {
        Debug.Log("S_Idleに入りました: 待機中...");
        // 待機アニメーション再生など
    }

    public override void Stay()
    {
        // 待機中の処理
    }

    public override void Exit()
    {
        Debug.Log("S_Idleを終了します");
    }
}
```

---

### サブステート付きステート（S_Attack）

**ファイル**: [S_Attack.cs](file:///c:/GitHub/Research/stateStrong/Assets/Laboratory/ProjectFenrirAI/StateMachine/Script/StateTest/S_Attack.cs)

親ステート`S_Attack`が3つのサブステートを管理：

| サブステート | 役割 | 時間 |
|---|---|---|
| `S_Attack_Start` | 予備動作 | 0.5秒 |
| `S_Attack_Exec` | 攻撃判定発生 | 0.2秒 |
| `S_Attack_End` | 硬直・残心 | 1.0秒 |

#### 親ステートでのサブステート登録

```
public override void Enter()
{
    // サブステートの登録
    AddSubState(new S_Attack_Start(owner));  // 0
    AddSubState(new S_Attack_Exec(owner));   // 1
    AddSubState(new S_Attack_End(owner));    // 2

    // 最初のサブステートへ遷移
    ChangeSubState(0);
}
```

**サブステートからの遷移**

```
// S_Attack_Start内
public override void Stay()
{
    m_Timer += Time.deltaTime;
    if (m_Timer >= m_Duration)
    {
        // 親ステート経由で次のサブステートへ
        GetParentState()?.ChangeSubState(1);
    }
}
```

**処理フロー図**

```
sequenceDiagram
    participant Owner as AITester
    participant Attack as S_Attack
    participant Start as S_Attack_Start
    participant Exec as S_Attack_Exec
    participant End as S_Attack_End

    Owner->>Attack: Enter()
    Attack->>Start: ChangeSubState(0)
    Start->>Start: Stay() (0.5秒経過)
    Start->>Exec: ChangeSubState(1)
    Exec->>Exec: Stay() (0.2秒経過)
    Exec->>End: ChangeSubState(2)
    End->>End: Stay() (1.0秒経過)
    End->>Owner: RevertToPreviousState()
    Owner->>Attack: Exit()
```

---

## 処理の流れ（ライフサイクル）

### 毎フレームの処理順序

```
flowchart TD
    A[StatefulObjectBase.Update] --> B[stateMachine.Update]
    B --> C{m_GlobalState != null?}
    C -->|Yes| D[m_GlobalState.Stay]
    C -->|No| E{m_CurrentState != null?}
    D --> E
    E -->|Yes| F[m_CurrentState.Stay]
    E -->|No| G[終了]
    F --> H{HasSubStates?}
    H -->|Yes| I[UpdateSubStates]
    H -->|No| G
    I --> G
```

### ステート変更時の処理順序

```
sequenceDiagram
    participant App as アプリケーション
    participant SM as StateMachine
    participant Old as 現在のステート
    participant New as 新しいステート

    App->>SM: ChangeState(newState)
    SM->>SM: 履歴にプッシュ
    SM->>Old: Exit()
```

```
    SM->>SM: m_CurrentState = newState
    SM->>New: Enter()
    SM-->>App: OnStateChanged イベント発火
```

---

## 変数命名規則

本プロジェクトでは以下の命名規則を適用します。

| 種類 | ルール | 例 |
|---|---|---|
| **ワールド変数（メンバー変数）** | `m_` + 大文字始まり | `m_Timer`, `m_Duration`, `m_CurrentState` |
| **ローカル変数** | `m_`なし + 小文字始まり | `timer`, `duration`, `prevName` |

例

```
public class ExampleState : State<AITester>
{
    // ワールド変数（メンバー変数）
    private float m_ElapsedTime;
    private bool m_IsActive;
    private int m_AttackCount;

    public ExampleState(AITester owner) : base(owner) { }

    public override void Stay()
    {
        // ローカル変数
        float deltaTime = Time.deltaTime;
        bool shouldAttack = m_ElapsedTime > 3.0f;

        if (shouldAttack)
        {
            int damage = CalculateDamage();  // ローカル変数
            m_AttackCount++;                  // メンバー変数をインクリメント
        }

        m_ElapsedTime += deltaTime;
    }
}
```

---

## 更新履歴

| 日付 | 内容 |
|---|---|
| 2026/01/16 | 初版作成 - ステートマシン構造解説ドキュメント |